

INTRODUCTION TO GPU PROGRAMMING WITH CUDA

Even if you don't play video games, you can be grateful to the game players, as their numbers have given rise to a class of highly powerful parallel processing devices—**graphics processing units** (GPUs). Yes, you program right on the video card in your computer, even though your program may have nothing to do with graphics or games.

15.1 Overview

The video game market is so lucrative that the industry has developed ever-faster GPUs, in order to handle ever-faster and ever-more visually detailed video games. These actually are parallel processing hardware devices, so around 2003 some people began to wonder if one might use them for parallel processing of nongraphics applications.

Originally this was cumbersome. One needed to figure out clever ways of mapping one's application to some kind of graphics problem, i.e. ways to disguising one's problem so that it appeared to be doing graphics computations. Though some high-level interfaces were developed to automate this transformation, effective coding required some understanding of graphics principles.

But current-generation GPUs separate out the graphics operations, and now consist of multiprocessor elements that run under the familiar shared-memory threads model. Thus they are easily programmable. Granted, effective coding still requires an intimate knowledge of the hardware, but at least it's (more or less) familiar hardware, not requiring knowledge of graphics.

Moreover, unlike a multicore machine, with the ability to run just a few threads

at one time, e.g. four threads on a quad core machine, GPUs can run *hundreds or thousands* of threads at once. There are various restrictions that come with this, but you can see that there is fantastic potential for speed here.

NVIDIA has developed the CUDA language as a vehicle for programming on their GPUs. It's basically just a slight extension of C, and has become very popular. More recently, the OpenCL language has been developed by Apple, AMD and others (including NVIDIA). It too is a slight extension of C, and it aims to provide a uniform interface that works with multicore machines in addition to GPUs. The same is true for another language, OpenACC. These languages are not (yet) in as broad use as CUDA, so our discussion here focuses on CUDA and NVIDIA GPUs.

Also, the discussion will focus on NVIDIA's Tesla line. This then led to the second generation, Fermi, Kepler, Pascal and so on. Unless otherwise stated, all statements here refer to Tesla, keeping things at the basic level.

15.2 Some Terminology

- A CUDA program consists of code to be run on the **host**, i.e. the CPU, and code to run on the **device**, i.e. the GPU.
- A function that is called by the host to execute on the device is called a **kernel**.
- Threads in an application are grouped into **blocks**. The entirety of blocks is called the **grid** of that application.

15.3 Example: Calculate Row Sums

Here's a sample program. And I've kept the sample simple: It just finds the sums of all the rows of a matrix.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <cuda.h>
4
5 // CUDA example: finds row sums of an integer matrix m
6
7 // find1elt() finds the rowsum of one row of the nxn matrix m, storing
8 // result in the corresponding position in the rowsum array rs; matrix
9 // stored as 1-dimensional, row-major order
```

```

10
11 __global__ void find1elt(int *m, int *rs, int n)
12 {
13     int rownum = blockIdx.x; // this thread will handle row # rownum
14     int sum = 0;
15     for (int k = 0; k < n; k++)
16         sum += m[rownum*n+k];
17     rs[rownum] = sum;
18 }
19
20 int main(int argc, char **argv)
21 {
22     int n = atoi(argv[1]); // number of matrix rows/cols
23     int *hm, // host matrix
24         *dm, // device matrix
25         *hrs, // host rowsums
26         *drs; // device rowsums
27     int msize = n * n * sizeof(int);
28     // size of matrix in bytes
29     // allocate space for host matrix
30     hm = (int *) malloc(msize);
31     // as a test, fill matrix with consecutive integers
32     int t = 0;
33     for (i = 0; i < n; i++) {
34         for (j = 0; j < n; j++) {
35             hm[i*n+j] = t++;
36         }
37     }
38     // allocate space for device matrix
39     cudaMalloc((void **)&dm,msize);
40     // copy host matrix to device matrix
41     cudaMemcpy(dm,hm,msize,cudaMemcpyHostToDevice);
42     // allocate host, device rowsum arrays
43     int rssize = n * sizeof(int);
44     hrs = (int *) malloc(rssize);
45     cudaMalloc((void **)&drs,rssize);
46     // set up parameters for threads structure
47     dim3 dimGrid(n,1); // n blocks
48     dim3 dimBlock(1,1,1); // 1 thread per block
49     // invoke the kernel
50     find1elt<<<dimGrid,dimBlock>>>(dm,drs,n);

```

```

50     // wait for kernel to finish
51     cudaThreadSynchronize();
52     // copy row vector from device to host
53     cudaMemcpy(hrs,drs,rssize,cudaMemcpyDeviceToHost);
54     // check results
55     if (n < 10) for(int i=0; i<n; i++) printf("%d\n",hrs[i]);
56     // clean up
57     free(hm);
58     cudaFree(dm);
59     free(hrs);
60     cudaFree(drs);
61 }

```

This is mostly C, with a bit of CUDA added here and there. Here's how the program works:

- Our **main()** runs on the host.
- Kernel functions are identified by **__global__ void**. They are called by the host and run on the device, thus serving as entries to the device.

We have only one kernel invocation here, but could have many, say with the output of one serving as input to the next.

- Other functions that will run on the device, called by functions running on the device, must be identified by **__device__**, e.g.

```
__device__ int sumvector(float *x, int n)
```

Of course, “will run on the device” means that the compiler will generate machine code for the device's architecture.

Note that unlike kernel functions, device functions can have return values, e.g. **int** above.

- When a kernel is called, each thread runs it. Each thread receives the same arguments.
- A “call” to a kernel is somewhat more than a simply call in C. Consider the code above,

```

dim3 dimGrid(n,1);
dim3 dimBlock(1,1,1);
find1elt<<<dimGrid,dimBlock>>>(dm,drs,n);

```

The first two statements set up the desired number of blocks per grid and the number of threads per block. The “call” then transmits this information to the GPU along with the ordinary arguments **dm** etc.

Due to the GPU’s heritage in physics, one has the option of making the grid look “two dimensional” to the programmer, and likewise make each block look “three dimensional.” This is just a programming convenience, which we will not pursue here.

- Each block and thread has an ID, stored in programmer-accessible structs **blockIdx** and **threadIdx**. We’ll discuss the details later, but for now, we’ll just note that here the statement

```
int rownum = blockIdx.x;
```

picks up the block number, which our code in this example uses to determine which row to sum.

- One calls **cudaMalloc()** on the host to dynamically allocate space on the device’s memory.¹ Execution of the statement

```
cudaMalloc((void **)&drs,rssize);
```

allocates space *on the device*, pointed to by **drs**, a variable in the *host’s* address space.

The space allocated by a **cudaMalloc()** call on the device is global to all kernels, and resides in the global memory of the device (details on memory types later).

One can also allocate device memory statically. For example, the statement

```
__device int z[100];
```

appearing outside any function definition would allocate space on device global memory, with scope global to all kernels. However, it is not accessible to the host.

- Data is transferred to and from the host and device memories via **cudaMemcpy()**. The fourth argument specifies the direction, e.g. **cudaMemcpyHostToDevice**, **cudaMemcpyDeviceToHost** or **cudaMemcpyDeviceToDevice**.
- Kernels return **void** values, so values are returned via a kernel’s arguments.

¹This function cannot be called from the device itself. However, **malloc()** is available from the device, and device memory allocated by it can be copied to the host. See the NVIDIA programming guide for details.

- Device functions (which we don't have in the above rowsums example) can return values. They are called only by kernel functions or other device functions.
- Note carefully that a call to the kernel doesn't block; it returns immediately. For that reason, the code above has a host barrier call, to avoid copying the results back to the host from the device before they're ready:

```
cudaThreadSynchronize();
```

On the other hand, if our code were to have another kernel call, say on the next line after

```
find1elt<<<dimGrid,dimBlock>>>(dm,drs,n);
```

and if some of the second call's input arguments were the outputs of the first call, there would be an implied barrier between the two calls; the second would not start execution before the first finished.

Calls like **cudaMemcpy()** do block until the operation completes.

There is also a thread barrier available for the threads themselves, *at the block level*. The call is

```
__syncthreads();
```

This can only be invoked by threads within a block, not across blocks. In other words, this is barrier synchronization within blocks.

- I've written the program so that each thread will handle one row of the matrix. I've chosen to store the matrix in one-dimensional form in row-major order, and the matrix is of size $n \times n$, so the loop

```
for (int k = 0; k < n; k++)
    sum += m[rownum*n+k];
```

will indeed traverse the n elements of row number **rownum**, and compute their sum. That sum is then placed in the proper element of the output array:

```
rs[rownum] = sum;
```

- After the kernel returns, the host must copy the result back from the device memory to the host memory, in order to access the results of the call.

15.4 Understanding the Hardware Structure

Scorecards, get your scorecards here! You can't tell the players without a scorecard—classic cry of vendors at baseball games

Know thy enemy—Sun Tzu, *The Art of War*

The enormous computational potential of GPUs cannot be unlocked without an intimate understanding of the hardware. This of course is a fundamental truism in the parallel processing world, but it is acutely important for GPU programming. This section presents an overview of the hardware.

15.4.1 Processing Units

A GPU consists of a large set of **streaming multiprocessors** (SMs). Since each SM is essentially a multicore machine in its own right, you might say the GPU is a multi-multiprocessor machine.

Each SM consists of a number of **streaming processors** (SPs), individual cores. The cores run threads, as with ordinary cores, but groups of threads within in an SM run in lockstep, to be explained below.

It is important to understand the motivation for this SM/SP hierarchy: Two threads located in different SMs cannot synchronize with each other in the barrier sense. Though this sounds like a negative at first, it is actually a great advantage, as the independence of threads in separate SMs means that the hardware can run faster. So, if the CUDA application programmer can write his/her algorithm so as to have certain independent chunks, and those chunks can be assigned to different SMs (we'll see how, shortly), then that's a "win."

Note basic word size is 32 bits. Newer devices are capable of double precision, etc., e.g. by declaring say,

```
float2 x;    // 64 bits
```

15.4.2 Thread Operation

GPU operation is highly threaded, and again, understanding of the details of thread operation is key to good performance.

SIMT Architecture

When you write a CUDA application program, you partition the threads into groups called **blocks**. The hardware will assign an entire block to a single SM,

though several blocks can run in the same SM. The hardware will then divide a block into **warps**, 32 threads to a warp. Knowing that the hardware works this way, the programmer controls the block size and the number of blocks, and in general **writes the code to take advantage of how the hardware works**.

The central point is that *all the threads in a warp run the code in lockstep*. During the machine instruction fetch cycle, the same instruction will be fetched for all of the threads in the warp. Then in the execution cycle, each thread will either execute that particular instruction or execute nothing. The execute-nothing case occurs in the case of branches; see below. This is the classical **single instruction, multiple data** (SIMD) pattern used in some early special-purpose computers such as the ILLIAC; here it is called **single instruction, multiple thread** (SIMT).

The syntactic details of grid and block configuration will be presented in Section [15.4.4](#).

The Problem of Thread Divergence

The SIMT nature of thread execution has major implications for performance. Consider what happens with if/then/else code. If some threads in a warp take the “then” branch and others go in the “else” direction, they cannot operate in lockstep. That means that some threads must wait while others execute. This renders the code at that point somewhat serial rather than parallel, a situation called **thread divergence**. As one CUDA Web tutorial points out, this can be a “performance killer.” (On the other hand, threads in the same block but in different warps can diverge with no problem.)

“OS in Hardware”

Each SM runs the threads on a timesharing basis, just like an operating system (OS). This timesharing is implemented in the hardware, though, not in software as in the OS case.

The “hardware OS” runs largely in analogy with an ordinary OS:

- A process in an ordinary OS is given a fixed-length timeslice, so that processes take turns running. In a GPU’s hardware OS, warps take turns running, with fixed-length timeslices.
- With an ordinary OS, if a process reaches an input/output operation, the OS suspends the process while I/O is pending, even if its turn is not up. The OS then runs some other process instead, so as to avoid wasting CPU cycles during the long period of time needed for the I/O.

With an SM, though, the analogous situation occurs when there is a long memory operation, to global memory; if a warp of threads needs to access global memory (including local memory; see below), the SM will schedule some other warp while the memory access is pending.

The hardware support for threads is extremely good. Each warp has its own set of registers, so a context switch does very little saving and restoring of context, quite a contrast to the OS case. Moreover, as noted above, the long latency of global memory may be solvable by having a lot of threads that the hardware can time-share to hide that latency; while one warp is fetching data from memory, another warp can be executing, thus not losing time due to the long fetch delay. For these reasons, CUDA programmers typically employ a large number of threads, each of which does only a small amount of work—again, quite a contrast to something like OpenMP, where coarser granularity is generally needed.

15.4.3 Memory Structure

The GPU memory hierarchy plays a key role in performance. Let's discuss the most important two types of memory first—shared and global.

Shared and Global Memory

Here is a summary:

type	shared	global
scope	glbl. to block	glbl. to app.
size	small	large
location	on-chip	off-chip
speed	blinding	molasses
lifetime	kernel	application
host access?	no	yes
cached?	no ²	no

In prose form:

- Shared memory: This memory is partitioned among all blocks in an SM. All threads in a block have access to this block's portion of the shared memory on that SM (but not access to the shared memory of other blocks). Access is very fast, both in terms of latency and bandwidth, as this memory is on-chip. It is declared inside the kernel, or in the kernel call (details below).

On the other hand, shared memory is small, 16K bytes per SM on the lower models, and the data stored in it are valid only for the life of the currently-

executing kernel. Also, shared memory cannot be accessed by the host. Note that the term *shared* only refers to the fact that it is shared among threads in the same block.

- Global memory: This is shared by all the threads in an entire application, and is persistent across kernel calls, throughout the life of the application, i.e. until the program running on the host exits. It is usually much larger than shared memory. It is accessible from the host. Pointers to global memory can (but do not have to) be declared outside the kernel.

On the other hand, global memory is off-chip and very slow, taking hundreds of clock cycles per access instead of just a few. As noted earlier, this can be ameliorated by exploiting latency hiding; we will elaborate on this in Section [15.4.3](#).

The reader should pause here and reread the above comparison between shared and global memories. *The key implication is that shared memory is used essentially as a programmer-managed cache.* Data will start out in global memory, but if a variable is to be accessed multiple times by the GPU code, it's probably better for the programmer to write code that copies it to shared memory, and then access the copy instead of the original. If the variable is changed and is to be eventually transmitted back to the host, the programmer must include code to copy it back to global memory.³

(Accesses to global and shared memory are done via half-warps, i.e. an attempt is made to do all memory accesses in a half-warp simultaneously. In that sense, only threads in a half-warp run simultaneously, but the full warp is *scheduled* to run contemporaneously by the hardware OS, the current instruction first running in one half-warp and then in the other. From the point of view of the programmer, the issue of half-warps is not important.)

The host can access global memory via `cudaMemcpy()`, as seen earlier. It cannot access shared memory. Here is a typical pattern:

```
__global__ void abckernel(int *abcglobalmem)
{
    __shared__ int abcsharedmem[100];
    // ... code to copy some of abcglobalmem to some of abcsharedmem
    // ... code for computation
    // ... code to copy some of abcsharedmem to some of abcglobalmem
}
```

Typically you would write the code so that each thread deals with its own portion of the shared data, e.g. its own portion of `abcsharedmem` and `abcglobalmem`

³Again, newer devices can relax this requirement somewhat; see Sec. [15.17](#)

above. However, all the threads in that block can read/write any element in `abcsharedmem`.

Shared memory consistency (recall Section 4.6) is sequential within a thread, but **relaxed** among threads in a block: A write by one thread is not guaranteed to be visible to the others in a block until `__syncthreads()` is called. On the other hand, writes by a thread *will* be visible to that same thread in subsequent reads without calling `__syncthreads()`. Among the implications of this is that if each thread writes only to portions of shared memory that are not read by other threads in the block, then `__syncthreads()` need not be called.

In the code fragment above, we allocated the shared memory through a C-style declaration:

```
__shared__ int abcsharedmem[100];
```

It is also possible to allocate shared memory in the kernel call, along with the block and thread configuration. Here is an example:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <cuda.h>
4
5 // CUDA example: illustrates kernel-allocated shared memory; does
6 // nothing useful, just copying an array from host to device global,
7 // then to device shared, doubling it there, then copying back to device
8 // global then host
9
10 __global__ void doubleit(int *dv, int n)
11 { extern __shared__ int sv[];
12   int me = threadIdx.x;
13   // threads share in copying dv to sv, with each thread copying one
14   // element
15   sv[me] = dv[me];
16   sv[me] = 2 * sv[me];
17   dv[me] = sv[me];
18 }
19
20 int main(int argc, char **argv)
21 {
22   int n = atoi(argv[1]); // number of matrix rows/cols
23   int *hv, // host array
24       *dv; // device array
```

```

25     int vsize = n * sizeof(int); // size of array in bytes
26     // allocate space for host array
27     hv = (int *) malloc(vsize);
28     // fill test array with consecutive integers
29     int t = 0,i;
30     for (i = 0; i < n; i++)
31         hv[i] = t++;
32     // allocate space for device array
33     cudaMalloc((void *)&dv,vsize);
34     // copy host array to device array
35     cudaMemcpy(dv,hv,vsize,cudaMemcpyHostToDevice);
36     // set up parameters for threads structure
37     dim3 dimGrid(1,1);
38     dim3 dimBlock(n,1,1); // all n threads in the same block
39     // invoke the kernel; third argument is amount of shared memory
40     doubleit<<<dimGrid,dimBlock,vsize>>>(dv,n);
41     // wait for kernel to finish
42     cudaThreadSynchronize();
43     // copy row array from device to host
44     cudaMemcpy(hv,dv,vsize,cudaMemcpyDeviceToHost);
45     // check results
46     if (n < 10) for(int i=0; i<n; i++) printf("%d\n",hv[i]);
47     // clean up
48     free(hv);
49     cudaFree(dv);
50 }

```

Here the variable **sv** is kernel allocated. It's declared in the statement

```
extern __shared__ int sv[];
```

but actually allocated during the kernel invocation

```
doubleit<<<dimGrid,dimBlock,vsize>>>(dv,n);
```

in that third argument within the chevrons, **vsize**.

Note that one can only directly declare one region of space in this manner. This has two implications:

- Suppose we have two `__device__` functions, each declared an `extern __shared__` array like this. Those two arrays will occupy the same place in memory!

- Suppose within one `__device__` function, we wish to have two **extern** `__shared__` arrays. We cannot do that literally, but we can share the space via subarrays, e.g.:

```
int *x = &sv[120];
```

would set up **x** as a subarray of **sv** above, starting at element 120.

One can also set up shared arrays of fixed length in the same code. Declare them before the variable-length one.

In our example above, the array **sv** is syntactically local to the function **doubleit()**, but is shared by all invocations of that function in the block, thus acting “global” to them in a sense. But the point is that it is not accessible from within *other* functions running in that block. In order to achieve the latter situation, a shared array can be declared outside any function.

Global-Memory Performance Issues

As noted, the latency (Section ??) for global memory is quite high, on the order of hundreds of clock cycles. However, the hardware attempts to ameliorate this problem in a couple of ways.

First, as mentioned earlier, if a warp has requested a global memory access that will take a long time, the hardware will schedule another warp to run while the first is waiting for the memory access to complete. This is an example of a common parallel processing technique called **latency hiding**.

Second, the bandwidth (Section ??) to global memory can be high, due to hardware actions called **coalescing**. This simply means that if the hardware sees that the threads in this half-warp (or at least the ones currently accessing global memory) are accessing consecutive words, the hardware can execute the memory requests in groups of up to 32 words at a time. This works because the memory is low-order interleaved (Section 4.2.1), and is true for both reads and writes.

The newer GPUs go even further, coalescing much more general access patterns, not just to consecutive words.

The programmer may be able to take advantage of coalescing, by a judicious choice of algorithms and/or by inserting padding into arrays (Section 4.2.2).

Shared-Memory Performance Issues

Shared memory is divided into banks, in a low-order interleaved manner (recall Section 4.2): Words with consecutive addresses are stored in consecutive banks,

mod the number of banks, i.e. wrapping back to 0 when hitting the last bank. If for instance there are 8 banks, addresses 0, 8, 16,... will be in bank 0, addresses 1, 9, 17,... will be in bank 1 and so on. (Actually, older devices have 16 banks, while newer ones have 32.) The fact that all memory accesses in a half-warp are attempted simultaneously implies that the best access to shared memory arises when the accesses are to different banks, just as for the case of global memory.

An exception occurs in **broadcast**. If all threads in the block wish to read from the same word in the same bank, the word will be sent to all the requestors simultaneously without conflict. However, if only some threads try to read the same word, there may or may not be a conflict, as the hardware chooses a bank for broadcast in some unspecified way.

As in the discussion of global memory above, we should write our code to take advantage of these structures.

The biggest performance issue with shared memory is its size, as little as 16K per SM in many GPU cards. And remember, this is divided up among the blocks on a given SM. If we have 4 blocks running on an SM, each one can only use $16K/4 = 4K$ bytes of shared memory.

Host/Device Memory Transfer Performance Issues

Copying data between host and device can be a major bottleneck. One way to ameliorate this is to use **cudaMallocHost()** instead of **malloc()** when allocating memory on the host. This sets up page-locked memory, meaning that it cannot be swapped out by the OS' virtual memory system. This allows the use of DMA hardware to do the memory copy, said to make **cudaMemcpy()** twice as fast.

Other Types of Memory

There are also other types of memory. Again, let's start with a summary:

type	registers	local	constant	texture
scope	single thread	single thread	glbl. to app.	glbl. to app.
location	device	device	host+device cache	host+device cache
speed	fast	molasses	fast if cache hit	fast if cache hit
lifetime	kernel	kernel	application	application
host access?	no	no	yes	yes
device access?	read/write	read/write	read	read

- **Registers:**

Each SM has a set of registers, much more numerous than in a CPU. Access to them is very fast, said to be slightly faster than to shared memory.

The compiler normally stores the local variables for a device function in registers, but there are exceptions. An array won't be placed in registers if the array is too large, or if the array has variable index values, such as

```
int z[20],i;
...
y = z[i];
```

Since registers are not indexable by the hardware, the compiler cannot allocate **z** to registers in this case. If on the other hand, the only code accessing **z** has constant indices, e.g. **z[8]**, the compiler may put **z** in registers.

- **Local memory:**

This is physically part of global memory, but is an area within that memory that is allocated by the compiler for a given thread. As such, it is slow, and accessible only by that thread. The compiler allocates this memory for local variables in a device function if the compiler cannot store them in registers. This is called **register spill**.

- **Constant memory:**

As the name implies, it's read-only from the device (read/write by the host), for storing values that will not be changed by device code. It is off-chip, thus potentially slow, but has a cache on the chip. At present, the size is 64K.

One designates this memory with `__constant__`, as a global variable in the source file. One sets its contents from the host via **cudaMemcpyToSymbol()**, whose (simple form for the) call is

```
cudaMemcpyToSymbol(var_name,pointer_to_source,number_bytes_copy,cudaMemcpyHostToDevice)
```

For example:

```
__constant__ int x; // not contained in any function

// host code
int y = 3;
cudaMemcpyToSymbol("x",&y,sizeof(int));
...

// device code
int z;
z = x;
```

Note again that the name Constant refers to the fact that device code cannot

change it. But host code certainly can change it between kernel calls. This might be useful in iterative algorithms like this:

```
/ host code
for 1 to number of iterations
    set Constant array x
    call kernel (do scatter op)
    cudaThreadSynchronize()
    do gather op, using kernel results to form new x

// device code
use x together with thread-specific data
return results to host
```

- **Texture:**

This is similar to constant memory, in the sense that it is read-only and cached. The difference is that the caching is two-dimensional. The elements **a[i][j]** and **a[i+1][j]** are far from each other in the global memory, but since they are “close” in a two-dimensional sense, they may reside in the same cache line.

15.4.4 Threads Hierarchy

Following the hardware, threads in CUDA software follow a hierarchy:

- The entirety of threads for an application is called a **grid**.
- A grid consists of one or more **blocks** of threads.
- Each block has its own ID within the grid, consisting of an “x coordinate” and a “y coordinate.”
- Likewise each thread has x, y and z coordinates within whichever block it belongs to.
- Just as an ordinary CPU thread needs to be able to sense its ID, e.g. in OpenMP by calling **omp_get_thread_num()**, CUDA threads need to do the same. A CUDA thread can access its block ID via the built-in variables **blockIdx.x** and **blockIdx.y**, and can access its thread ID within its block via **threadIdx.x**, **threadIdx.y** and **threadIdx.z**.
- The programmer specifies the grid size (the numbers of rows and columns of blocks within a grid) and the block size (numbers of rows, columns and layers of threads within a block). In the first example above, this was done by the code


```
dim3 dimGrid(n,1);
dim3 dimBlock(1,1,1);
find1elt<<<dimGrid,dimBlock>>>(dm,drs,n);
```

Here the grid is specified to consist of n ($n \times 1$) blocks, and each block consists of just one ($1 \times 1 \times 1$) thread. (As noted earlier, this is just an example, very inefficient but simple for introductory purposes.)

That last line is of course the call to the kernel. As you can see, CUDA extends C syntax to allow specifying the grid and block sizes. CUDA will store this information in structs of type **dim3**, in this case our variables **gridDim** and **blockDim**, accessible to the programmer, again with member variables for the various dimensions, e.g. **blockDim.x** for the size of the x dimension for the number of threads per block.

- All threads in a block run in the same SM, though more than one block might be on the same SM.
- The “coordinates” of a block within the grid, and of a thread within a block, are merely abstractions. If for instance one is programming computation of heat flow across a two-dimensional slab, the programmer may find it clearer to use two-dimensional IDs for the threads. *But this does not correspond to any physical arrangement in the hardware.*

As noted, the motivation for the two-dimensional block arrangement is to make coding conceptually simpler for the programmer if he/she is working an application that is two-dimensional in nature.

For example, in a matrix application one’s parallel algorithm might be based on partitioning the matrix into rectangular submatrices (**tiles**), as we’ll do in Section [20.2](#). In a small example there, the matrix

$$A = \begin{pmatrix} 1 & 5 & 12 \\ 0 & 3 & 6 \\ 4 & 8 & 2 \end{pmatrix} \quad (15.1)$$

is partitioned as

$$A = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix}, \quad (15.2)$$

where

$$A_{00} = \begin{pmatrix} 1 & 5 \\ 0 & 3 \end{pmatrix}, \quad (15.3)$$

$$A_{01} = \begin{pmatrix} 12 \\ 6 \end{pmatrix}, \quad (15.4)$$

$$A_{10} = \begin{pmatrix} 4 & 8 \end{pmatrix} \quad (15.5)$$

and

$$A_{11} = \begin{pmatrix} 2 \end{pmatrix}. \quad (15.6)$$

We might then have one block of threads handle A_{00} , another block handle A_{01} and so on. CUDA's two-dimensional ID system for blocks makes life easier for programmers in such situations.

15.4.5 What's NOT There

We're not in Kansas anymore, Toto—character Dorothy Gale in *The Wizard of Oz*

It looks like C, it feels like C, and for the most part, it *is* C. But in many ways, it's quite different from what you're used to:⁴

- You don't have access to the C library (the library consists of host machine language, after all). There are special versions of math functions, however, e.g. `__sin()`.
- No stack. Functions are essentially inlined, rather than their calls being handled by pushes onto a stack.
- No pointers to functions.

15.5 Synchronization, Within and Between Blocks

As mentioned, a barrier for the threads in the same block is available by calling `__syncthreads()`. Note carefully that if one thread writes a variable to memory (of whatever kind) and another then reads that variable, one must call this function (from both threads) in order to get the latest value. Keep in mind that

⁴As noted, the statements below apply to the Tesla series.

within a block, different warps will run at different times, making synchronization vital.

Remember too that threads across blocks cannot sync with each other in this manner. There are, though, several **atomic** operations—read/modify/write actions that a thread can execute without **pre-emption**, i.e. without interruption—available on both global and shared memory. For example, **atomicAdd()** performs a fetch-and-add operation, as described in Section 4.4.3 of this book. The call is

```
atomicAdd(address of integer variable,inc);
```

where **address of integer variable** is the address of the (device) variable to add to, and **inc** is the amount to be added. The return value of the function is the value originally at that address before the operation.

There are also **atomicExch()** (exchange the two operands), **atomicCAS()** (if the first operand equals the second, replace the first by the third), **atomicMin()**, **atomicMax()**, **atomicAnd()**, **atomicOr()**, and so on.

As with compare-and-exchange operations on other machines, we can use it to implement locks, e.g.⁵

```
__device__ void lock(int* lockVar) {
    while (atomicCAS(lockVar, 0, 1) != 0) { ; }
}

__device__ void unlock(int* lockVar) {
    atomicExch(lockVar, 0);
}
```

Use **-arch=sm_11** when compiling, e.g.

```
nvcc -g -G yoursrc.cu -arch=sm_11
```

Though a barrier could in principle be constructed from the atomic operations, its overhead would be quite high. In earlier models that was near a microsecond, and though that problem has been ameliorated in more recent models, implementing a barrier in this manner would not be not much faster than attaining interblock synchronization by returning to the host and calling **cudaThreadSynchronize()** there. Recall that the latter *is* a possible way to implement a barrier, since global memory stays intact in between kernel calls, but again, it would be slow.

⁵Modeled on <http://cs.wm.edu/~finlayson/class/fall14/cpsc425/notes/22-cuda-sync.html>.

So, what if synchronization is really needed? This is the case, for instance, for iterative algorithms, where all threads must wait at the end of each iteration.

If you have a small problem, maybe you can get satisfactory performance by using just one block, thus enabling the use of `__syncthreads()`. You'll have to use a larger granularity, i.e. more work assigned to each thread. But using just one block means you're using only one SM, thus only a fraction of the potential power of the GPU.

If you use multiple blocks, though, your only feasible options for synchronization are to either use the global atomic operations or rely on returns to the host, where synchronization occurs via `cudaThreadSynchronize()`. You would then have the situation outlined in the discussion of Constant memory in Section [15.4.3](#).

15.6 More on the Blocks/Threads Tradeoff

Resource size considerations must be kept in mind when you design your code and your grid configuration. In particular, note the following:

- Each block in your code is assigned to some SM. It will be tied to that SM during the entire execution of your kernel, though of course it will not constantly be running during that time.
- If there are more blocks than can be accommodated by all the SMs, then some blocks will need to wait for assignment; when a block finishes, that block's resources, e.g. shared memory, can now be assigned to a waiting block.
- The programmer has no control over which block is assigned to which SM.
- Within a block, threads execute by the warp, 32 threads. At any give time, the SM is running one warp, chosen by the GPU OS.
- The GPU has a limit on the number of threads that can run on a single block, typically 512, and on the total number of threads running on an SM, 786.
- If a block contains fewer than 32 threads, only part of the processing power of the SM it's running on will be used. So block size should normally be at least 32. Moreover, for the same reason, block size should ideally be a multiple of 32.
- If your code makes use of shared memory, larger block size may be the better, so as to have more shared memory available per block. . On the

other hand, the larger the block size, the longer the time it will take for barrier synchronization.

- We want to use the full power of the GPU, with its many SMs, thus implying a need to use at least as many blocks as there are SMs (which may require smaller blocks).
- Moreover, due to the need for latency hiding in memory access, we want to have lots of warps, so that some will run while others are doing memory access. This argues for a larger block size.
- Two threads doing unrelated work, or the same work but with many if/elses, would cause a lot of thread divergence if they were in the same block. This argues for a smaller block size, and/or a judicious arrangement of code on the programmer's part.
- A commonly-cited rule of thumb is to have between 128 and 256 threads per block.

Though there is a limit on the number of blocks, this limit will be much larger than the number of SMs. So, you may have multiple blocks running on the same SM. Since execution is scheduled by the warp anyway, there appears to be no particular drawback to having more than one block on the same SM.

15.7 Hardware Requirements, Installation, Compilation, Debugging

You do need a suitable NVIDIA video card. There is a list at http://www.nvidia.com/object/cuda_gpus.html; see also the Wikipedia entry, http://en.wikipedia.org/wiki/CUDA#Supported_GPUs. If you have a Linux system, run `lspci` to determine what kind you have.⁶

Download the CUDA toolkit from NVIDIA. Just plug “CUDA download” into a Web search engine to find the site. Install as directed.

You'll need to set your search and library paths to include the CUDA **bin** and **lib** directories.

To compile **x.cu** (and yes, use the **.cu** suffix), type

```
$ nvcc -g -G x.cu
```

⁶It might be in **/sbin**.

The **-g -G** options are for setting up debugging, the first for host code, the second for device code. You may also need to specify

```
-I/your_CUDA_include_path
```

to pick up the file **cuda.h**. Run the code as you normally would.

You may need to take special action to set your library path properly. For example, on Linux machines, set the environment variable **LD_LIBRARY_PATH** to include the CUDA library.

To determine the limits, e.g. maximum number of threads, for your device, use code like this:

```
cudaDeviceProp Props;
cudaGetDeviceProperties(&Props,0);
```

The 0 is for device 0, assuming you only have one device. The return value of **cudaGetDeviceProperties()** is a complex C struct whose components are listed at http://developer.download.nvidia.com/compute/cuda/2_3/toolkit/docs/online/group__CUDART__DEVICE_g5aa4f47938af8276f08074d09b7d520c.html.

Here's a simple program to check some of the properties of device 0:

```
1 #include <cuda.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     cudaDeviceProp Props;
7     cudaGetDeviceProperties( &Props,0);
8
9     printf("shared mem: %d\n", Props.sharedMemPerBlock);
10    printf("max threads/block: %d\n", Props.maxThreadsPerBlock);
11    printf("max blocks: %d\n", Props.maxGridSize[0]);
12    printf("total Const mem: %d\n", Props.totalConstMem);
13 }
```

Under older versions of CUDA, such as 2.3, one can debug using GDB as usual. You must compile your program in emulation mode, using the **-deviceemu** command-line option. This is no longer available as of version 3.2. CUDA also includes a special version of GDB, CUDA-GDB (invoked as **cuda-gdb**) for real-time debugging. However, on Unix-family platforms it runs only if X11 is not

running. Short of dedicating a machine for debugging, you may find it useful to install a version 2.3 in addition to the most recent one to use for debugging.

15.8 Example: Improving the Row Sums Program

The issues involving coalescing in Section [15.4.3](#) would suggest that our rowsum code might run faster with column sums, to take advantage of the memory banking. (So the user would either need to take the transpose first, or have his code set up so that the matrix is in transpose form to begin with.) As two threads in the same half-warp march down adjoining columns in lockstep, they will always be accessing adjoining words in memory.

So, I modified the program accordingly (not shown), and compiled the two versions, as **rs** and **cs**, the row- and column-sum versions of the code, respectively.

This did produce a small improvement (confirmed in subsequent runs, needed in any timing experiment):

```
pc5:~/CUDA% time rs 20000
2.585u 1.753s 0:04.54 95.3%    0+0k 7104+0io 54pf+0w
pc5:~/CUDA% time cs 20000
2.518u 1.814s 0:04.40 98.1%    0+0k 536+0io 5pf+0w
```

But let's compare it to a version running only on the CPU,

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // non-CUDA example:  finds col sums of an integer matrix m
5
6 // find1elt() finds the colsum of one col of the nxn matrix m, storing
7 // result in the corresponding position in the colsum array cs; matrix
8 // stored as 1-dimensional, row-major order
9
10 void find1elt(int *m, int *cs, int n)
11 {
12     int sum=0;
13     int topofcol;
14     int col,k;
15     for (col = 0; col < n; col++) {
16         topofcol = col;
17         sum = 0;
18         for (k = 0; k < n; k++)
19             sum += m[topofcol+k*n];
```

```

20         cs[col] = sum;
21     }
22 }
23
24 int main(int argc, char **argv)
25 {
26     int n = atoi(argv[1]); // number of matrix cols/cols
27     int *hm, // host matrix
28         *hcs; // host colsums
29     int msize = n * n * sizeof(int);
30     // size of matrix in bytes
31     // allocate space for host matrix
32     hm = (int *) malloc(msize);
33     // as a test, fill matrix with consecutive integers
34     int t = 0,i,j;
35     for (i = 0; i < n; i++) {
36         for (j = 0; j < n; j++) {
37             hm[i*n+j] = t++;
38         }
39     }
40     int cssize = n * sizeof(int);
41     hcs = (int *) malloc(cssize);
42     find1elt(hm,hcs,n);
43     if (n < 10) for(i=0; i<n; i++) printf("%d\n",hcs[i]);
44     // clean up
45     free(hm);
46     free(hcs);
47 }

```

How fast does this non-CUDA version run?

```

pc5:~/CUDA% time csc 20000
61.110u 1.719s 1:02.86 99.9%    0+0k 0+0io 0pf+0w

```

Very impressive! No wonder people talk of CUDA in terms like “a supercomputer on our desktop.” And remember, this includes the time to copy the matrix from the host to the device (and to copy the output array back). And we didn’t even try to optimize thread configuration, memory coalescing and bank usage, making good use of memory hierarchy, etc.⁷

⁷Neither has the CPU-only version of the program been optimized. As pointed out by Bill Hsu, the row-major version of that program should run faster than the column-major one, due to cache considerations.

On the other hand, remember that this is an “embarrassingly parallel” application, and in many applications we may have to settle for a much more modest increase, and work harder to get it.

15.9 Example: Finding the Mean Number of Mutual Outlinks

As in Sections ?? and [6.12](#) consider a network graph of some kind, such as Web links. For any two vertices, say any two Web sites, we might be interested in mutual outlinks, i.e. outbound links that are common to two Web sites. The CUDA code below finds the mean number of mutual outlinks, among all pairs of sites in a set of Web sites.

```
1 #include <cuda.h>
2 #include <stdio.h>
3
4 // CUDA example: finds mean number of mutual outlinks, among all pairs
5 // of Web sites in our set; in checking all (i,j) pairs, thread k will
6 // handle all i such that i mod totth = k, where totth is the number of
7 // threads
8
9 // procpairs() processes all pairs for a given thread
10 __global__ void procpairs(int *m, int *tot, int n)
11 { int totth = gridDim.x * blockDim.x,
  // total number of threads
12     // need to find my thread number among the totality of all
13     // threads in all blocks
14     me = blockIdx.x * blockDim.x + threadIdx.x;
15     int i,j,k,sum = 0;
16     for (i = me; i < n; i += totth) {
  // do various rows i
17         for (j = i+1; j < n; j++) { // do all rows j > i
18             for (k = 0; k < n; k++)
19                 sum += m[n*i+k] * m[n*j+k];
20         }
21     }
22     atomicAdd(tot,sum);
23 }
24
25 int main(int argc, char **argv)
```

```

26 {   int n = atoi(argv[1]), // number of vertices
27       nblk = atoi(argv[2]); // number of blocks
28     int *hm, // host matrix
29         *dm, // device matrix
30         htot, // host grand total
31         *dtot; // device grand total
32     int msize = n * n * sizeof(int);
    // size of matrix in bytes
33     // allocate space for host matrix
34     hm = (int *) malloc(msize);
35     // as a test, fill matrix with random 1s and 0s
36     int i,j;
37     for (i = 0; i < n; i++) {
38         hm[n*i+i] = 0;
39         for (j = 0; j < n; j++) {
40             if (j != i) hm[i*n+j] = rand() % 2;
41         }
42     }
43     // allocate space for device matrix
44     cudaMalloc((void **)&dm,msize);
45     // copy host matrix to device matrix
46     cudaMemcpy(dm,hm,msize,cudaMemcpyHostToDevice);
47     htot = 0;
48     // set up device total and initialize it
49     cudaMalloc((void **)&dtot,sizeof(int));
50     cudaMemcpy(dtot,&htot,sizeof(int),cudaMemcpyHostToDevice);
51     // set up parameters for threads structure
52     dim3 dimGrid(nblk,1);
53     dim3 dimBlock(192,1,1);
54     // invoke the kernel
55     procpairs<<<dimGrid,dimBlock>>>(dm,dtot,n);
56     // wait for kernel to finish
57     cudaThreadSynchronize();
58     // copy total from device to host
59     cudaMemcpy(&htot,dtot,sizeof(int),cudaMemcpyDeviceToHost);
60     // check results
61     if (n <= 15) {
62         for (i = 0; i < n; i++) {
63             for (j = 0; j < n; j++)
64                 printf("%d ",hm[n*i+j]);
65             printf("\n");

```

```

66     }
67 }
68 printf("mean = %f\n",htot/float((n*(n-1))/2));
69 // clean up
70 free(hm);
71 cudaFree(dm);
72 cudaFree(dtot);
73 }

```

Again we've used the method in Section ?? to partition the various pairs (i,j) to the different threads. Note the use of **atomicAdd()**.

The above code is hardly optimal. The reader is encouraged to find improvements.

15.10 Example: Finding Prime Numbers

The code below finds all the prime numbers from 2 to **n**.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <cuda.h>
4
5 // CUDA example:  illustration of shared memory allocation at run time
6 // finds primes using classical Sieve of Erathosthenes:
   make list of
7 // numbers 2 to n, then cross out all multiples of 2 (but not 2 itself
8 // then all multiples of 3, etc.; whatever is left over is prime; in c
9 // array, 1 will mean "not crossed out" and 0 will mean "crossed out"
10
11 // IMPORTANT NOTE: uses shared memory, in a single block, without
12 // rotating parts of array in and out of shared memory; thus limited t
13 // n <= 4000 if have 16K shared memory, and an inefficient use of the
14 // GPU in any case
15
16 // initialize sprimes, 1s for the odds, 0s for the evens; see sieve()
17 // for the nature of the arguments
18 __device__ void initsp(int *sprimes, int n, int nth, int me)
19 {
20     int chunk,startsetsp,endsetsp,val,i;
21     sprimes[2] = 1;
22     // determine sprimes chunk for this thread to init
23     chunk = (n-1) / nth;

```

```

24     startsetsp = 2 + me*chunk;
25     if (me < nth-1) endsetsp = startsetsp + chunk - 1;
26     else endsetsp = n;
27     // now do the init
28     val = startsetsp % 2;
29     for (i = startsetsp; i <= endsetsp; i++) {
30         sprimes[i] = val;
31         val = 1 - val;
32     }
33     // make sure sprimes up to date for all
34     __syncthreads();
35 }
36
37 // copy sprimes back to device global memory; see sieve() for the nature
38 // of the arguments
39 __device__ void cpytoglb(int *dprimes, int *sprimes, int n, int nth, int nth1)
40 {
41     int startcpy, endcpy, chunk, i;
42     chunk = (n-1) / nth;
43     startcpy = 2 + me*chunk;
44     if (me < nth-1) endcpy = startcpy + chunk - 1;
45     else endcpy = n;
46     for (i = startcpy; i <= endcpy; i++) dprimes[i] = sprimes[i];
47     __syncthreads();
48 }
49
50 // finds primes from 2 to n, storing the information in dprimes, with
51 // dprimes[i] being 1 if i is prime, 0 if composite; nth is the number
52 // of threads (threadDim somehow not recognized)
53 __global__ void sieve(int *dprimes, int n, int nth)
54 {
55     extern __shared__ int sprimes[];
56     int me = threadIdx.x;
57     int nth1 = nth - 1;
58     // initialize sprimes array, 1s for odds, 0 for evens
59     initsp(sprimes, n, nth, me);
60     // "cross out" multiples of various numbers m, with each thread doing
61     // a chunk of m's; always check first to determine whether m has
62     // already been found to be composite; finish when m*m > n
63     int maxmult, m, startmult, endmult, chunk, i;
64     for (m = 3; m*m <= n; m++) {

```

```

65         if (sprimes[m] != 0) {
66             // find largest multiple of m that is <= n
67             maxmult = n / m;
68             // now partition 2,3,...,maxmult among the threads
69             chunk = (maxmult - 1) / nth;
70             startmult = 2 + me*chunk;
71             if (me < nth-1) endmult = startmult + chunk - 1;
72             else endmult = maxmult;
73         }
74         // OK, cross out my chunk
75         for (i = startmult; i <= endmult; i++) sprimes[i*m] = 0;
76     }
77     __syncthreads();
78     // copy back to device global memory for return to host
79     cpytoglb(dprimes,sprimes,n,nth,me);
80 }
81
82 int main(int argc, char **argv)
83 {
84     int n = atoi(argv[1]), // will find primes among 1,...,n
85         nth = atoi(argv[2]); // number of threads
86     int *hprimes, // host primes list
87         *dprimes; // device primes list
88     int psize = (n+1) * sizeof(int);
89     // size of primes lists in bytes
90     // allocate space for host list
91     hprimes = (int *) malloc(psize);
92     // allocate space for device list
93     cudaMalloc((void *)&dprimes,psize);
94     dim3 dimGrid(1,1);
95     dim3 dimBlock(nth,1,1);
96     // invoke the kernel, including a request to allocate shared memory
97     sieve<<<dimGrid,dimBlock,psize>>>(dprimes,n,nth);
98     // check whether we asked for too much shared memory
99     cudaError_t err = cudaGetLastError();
100     if(err != cudaSuccess) printf("%s\n",cudaGetErrorString(err));
101     // wait for kernel to finish
102     cudaThreadSynchronize();
103     // copy list from device to host
104     cudaMemcpy(hprimes,dprimes,psize,cudaMemcpyDeviceToHost);
105     // check results

```

```

105     if (n <= 1000) for(int i=2; i<=n; i++)
106         if (hprimes[i] == 1) printf("%d\n",i);
107     // clean up
108     free(hprimes);
109     cudaFree(dprimes);
110 }

```

This code has been designed with some thought as to memory speed and thread divergence. Ideally, we would like to use device shared memory if possible, and to exploit the lockstep, SIMD nature of the hardware.

The code uses the classical Sieve of Erathosthenes, “crossing out” multiples of 2, 3, 5, 7 and so on to get rid of all the composite numbers. However, the code here differs from that in Section ??, even though both programs use the Sieve of Erathosthenes.

Say we have just two threads, A and B. In the earlier version, thread A might cross out all multiples of 19 while B handles multiples of 23. In this new version, thread A deals with only some multiples of 19 and B handles the others for 19. Then they both handle their own portions of multiples of 23, and so on. The thinking here is that the second version will be more amenable to lockstep execution, thus causing less thread divergence.

Thus in this new version, each thread handles a chunk of multiples of the given prime. Note the contrast of this with many CUDA examples, in which each thread does only a small amount of work, such as computing a single element in the product of two matrices.

In order to enhance memory performance, this code uses device shared memory. All the “crossing out” is done in the shared memory array **sprimes**, and then when we are all done, that is copied to the device global memory array **dprimes**, which is in turn copies to host memory. By the way, note that the amount of shared memory here is determined dynamically.

However, device shared memory consists only of 16K bytes, which would limit us here to values of **n** up to about 4000. Moreover, by using just one block, we are only using a small part of the CPU. Extending the program to work for larger values of **n** would require some careful planning if we still wish to use shared memory.

Also, it is quite important to note that in this simple version, we are using only one block, thus only one SM, thus only a fraction of the GPU. This would have to be remedied for the best performance.

15.11 Example: Finding Cumulative Sums

Here we wish to compute cumulative sums. For instance, if the original array is (3,1,2,0,3,0,1,2), then it is changed to (3,4,6,6,9,9,10,12).

(Note: This is a special case of the *prefix scan* problem, covered in Chapter ??.)

The general plan is for each thread to operate on one chunk of the array. A thread will find cumulative sums for its chunk, and then adjust them based on the high values of the chunks that precede it. In the above example, for instance, say we have 4 threads. The threads will first produce (3,4), (2,2), (3,3) and (1,3). Since thread 0 found a cumulative sum of 4 in the end, we must add 4 to each element of (2,2), yielding (6,6). Thread 1 had found a cumulative sum of 2 in the end, which together with the 4 found by thread 0 makes 6. Thus thread 2 must add 6 to each of its elements, i.e. add 6 to (3,3), yielding (9,9). The case of thread 3 is similar.

Below is code for the special case of a single block; note disclaimers above.

```
1 // for this simple illustration, it is assumed that the code runs in
2 // just one block, and that the number of threads evenly divides n
3
4 // improvements that could be made:
5 //     1. change to multiple blocks, to try to use all SMs
6 //     2. possibly use shared memory
7 //     3. have each thread work on staggered elements of dx, rather th
8 //         on contiguous ones, to get more efficient bank access
9
10 #include <cuda.h>
11 #include <stdio.h>
12
13 __global__ void cumulker(int *dx, int n)
14 {
15     int me = threadIdx.x;
16     int csize = n / blockDim.x;
17     int start = me * csize;
18     int i,j,base;
19     for (i = 1; i < csize; i++) {
20         j = start + i;
21         dx[j] = dx[j-1] + dx[j];
22     }
23     __syncthreads();
24     if (me > 0) {
```

```

25     base = 0;
26     for (j = 0; j < me; j++)
27         base += dx[(j+1)*csize-1];
28 }
29 if (me > 0) {
30     for (i = start; i < start + csize; i++)
31         dx[i] += base;
32 }
33 }
34
35
36 int main(int argc, char **argv)
37 {
38     int n = atoi(argv[1]), // length of array
39         nth = atoi(argv[2]); // number of threads
40     int *ha, // host array
41         *da, // device array
42         nint = n * sizeof(int);
43     ha = (int *) malloc(nint);
44     // test example
45     for (int i = 0; i < n; i++) ha[i] = i*i % 5;
46     if (n < 100) for(int i=0; i<n; i++) printf("%d ",ha[i]);
47     printf("\n");
48     cudaMalloc((void **)&da,nint);
49     cudaMemcpy(da,ha,nint,cudaMemcpyHostToDevice);
50     dim3 dimGrid(1,1);
51     dim3 dimBlock(n/nth,1,1);
52     cumulker<<<dimGrid,dimBlock>>>(da,n);
53     cudaThreadSynchronize();
54     cudaMemcpy(ha,da,nint,cudaMemcpyDeviceToHost);
55     if (n < 100) for(int i=0; i<n; i++) printf("%d ",ha[i]);
56     printf("\n");
57     free(ha);
58     cudaFree(da);
59 }

```


15.12 When Is It Advantageous to Use Shared Memory

Shared memory only helps if we are doing multiple accesses to the data. If for instance our code does a single read and a single write to an element of an array, then transferring it back and forth between global and shared memory isn't worthwhile.

Would the cumulative-sums program in Section 15.11 benefit from the use of shared memory? (Put aside the fact that the code runs in just one block, making use of just a sliver of the machine.) The answer appears to be that a modest improvement might be obtained. Each thread (except the first) reads many elements of **dx** twice, some of them three times. There are also writes.

The case of the prime-finder program in Section 15.10 is less clear, and probably quite dependent on whether we are using the more advanced GPUs, which feature at least some L1 cache space.

15.13 Example: Transforming an Adjacency Matrix

Below is a CUDA version of the code in Section 6.13. Note that in this code, there are two kernel calls.

```
1 // CUDA example
2
3 // takes a graph adjacency matrix for a directed graph, and converts it
4 // to a 2-column matrix of pairs (i,j), meaning an edge from vertex i
5 // vertex j; the output matrix must be in lexicographical order
6
7 // not claimed efficient, either in speed or in memory usage
8
9 #include <cuda.h>
10 #include <stdio.h>
11
12 // needs -lrt link flag for C++
13 #include <time.h>
14 float timediff(struct timespec t1, struct timespec t2)
15 { if (t1.tv_nsec > t2.tv_nsec) {
16     t2.tv_sec -= 1;
17     t2.tv_nsec += 1000000000;
```

```

18     }
19     return t2.tv_sec-t1.tv_sec + 0.000000001 * (t2.tv_nsec-t1.tv_nsec);
20 }
21
22 // tgkernel1() finds the number of 1s to be handled by a thread, used
23 // to determine where in the output matrix a thread writes its portion
24
25 // arguments:
26 //     dadjm:  the adjacency matrix (NOT assumed symmetric), 1 for edge
27 //             otherwise; note: matrix is overwritten by the function
28 //     n:      number of rows and columns of adjm
29 //     dcounts: output array, counts of 1s
30
31 __global__ void tgkernel1(int *dadjm, int n, int *dcounts)
32 {
33     int tot1s,j;
34     // need to find my thread number among the totality of all
35     // threads in all blocks
36     int me = blockDim.x * blockIdx.x + threadIdx.x;
37     tot1s = 0;
38     for (j = 0; j < n; j++) {
39         if (dadjm[n*me+j] == 1) {
40             dadjm[n*me+tot1s++] = j;
41         }
42     }
43     dcounts[me] = tot1s;
44 }
45
46 // tgkernel2() has the given thread write its rows into the output
47 // matrix
48
49 __global__ void tgkernel2(int *dadjm, int n,
50     int *dcounts, int *dstarts, int *doutm)
51 {
52     int outrow,num1si,j;
53     int me = blockDim.x * blockIdx.x + threadIdx.x;
54     // fill in this thread's portion of doutm
55     outrow = dstarts[me];
56     num1si = dcounts[me];
57     if (num1si > 0) {
58         for (j = 0; j < num1si; j++) {
59             doutm[2*outrow+2*j] = me;
60             doutm[2*outrow+2*j+1] = dadjm[n*me+j];

```

```

59     }
60 }
61 }
62
63 // replaces counts by cumulative counts
64 void cumulcounts(int *c, int *s, int n)
65 { int i;
66   s[0] = 0;
67   for (i = 1; i < n; i++) {
68     s[i] = s[i-1] + c[i-1];
69   }
70 }
71
72 int *transgraph(int *hadjm, int n, int *nout, int gsize, int bsize)
73 { int *dadjm; // device adjacency matrix
74   int *houtm; // host output matrix
75   int *doutm; // device output matrix
76   int *hcounts; // host counts vector
77   int *dcounts; // device counts vector
78   int *hstarts; // host starts vector
79   int *dstarts; // device starts vector
80   hcounts = (int *) malloc(n*sizeof(int));
81   hstarts = (int *) malloc(n*sizeof(int));
82   cudaMalloc((void **)&dadjm, n*n*sizeof(int));
83   cudaMalloc((void **)&dcounts, n*sizeof(int));
84   cudaMalloc((void **)&dstarts, n*sizeof(int));
85   houtm = (int *) malloc(n*n*sizeof(int));
86   cudaMalloc((void **)&doutm, n*n*sizeof(int));
87   cudaMemcpy(dadjm, hadjm, n*n*sizeof(int), cudaMemcpyHostToDevice);
88   dim3 dimGrid(gsize, 1);
89   dim3 dimBlock(bsize, 1, 1);
90   // calculate counts and starts first
91   tgkernel1<<<dimGrid, dimBlock>>>(dadjm, n, dcounts);
92   cudaMemcpy(hcounts, dcounts, n*sizeof(int), cudaMemcpyDeviceToHost);
93   cumulcounts(hcounts, hstarts, n);
94   *nout = hstarts[n-1] + hcounts[n-1];
95   cudaMemcpy(dstarts, hstarts, n*sizeof(int), cudaMemcpyHostToDevice);
96   tgkernel2<<<dimGrid, dimBlock>>>(dadjm, n, dcounts, dstarts, doutm);
97   cudaMemcpy(houtm, doutm, 2>(*nout)*sizeof(int), cudaMemcpyDeviceToHost);
98   free(hcounts);
99   free(hstarts);

```

```

100     cudaFree(dadjm);
101     cudaFree(dcounts);
102     cudaFree(dstarts);
103     return houtm;
104 }
105
106 int main(int argc, char **argv)
107 {   int i,j;
108     int *adjm; // host adjacency matrix
109     int *outm; // host output matrix
110     int n = atoi(argv[1]);
111     int gsize = atoi(argv[2]);
112     int bsize = atoi(argv[3]);
113     int nout;
114     adjm = (int *) malloc(n*n*sizeof(int));
115     for (i = 0; i < n; i++)
116         for (j = 0; j < n; j++)
117             if (i == j) adjm[n*i+j] = 0;
118             else adjm[n*i+j] = rand() % 2;
119     if (n < 10) {
120         printf("adjacency matrix: \n");
121         for (i = 0; i < n; i++) {
122             for (j = 0; j < n; j++) printf("%d ",adjm[n*i+j]);
123             printf("\n");
124         }
125     }
126
127     struct timespec bgn,nd;
128     clock_gettime(CLOCK_REALTIME, &bgn);
129
130     outm = transgraph(adjm,n,&nout,gsize,bsize);
131     printf("num rows in out matrix = %d\n",nout);
132     if (nout < 50) {
133         printf("out matrix: \n");
134         for (i = 0; i < nout; i++)
135             printf("%d %d\n",outm[2*i],outm[2*i+1]);
136     }
137
138     clock_gettime(CLOCK_REALTIME, &nd);
139     printf("%f\n",timediff(bgn,nd));
140 }

```

15.14 Error Checking

Every CUDA call (except for kernel invocations) returns an error code of type **cudaError_t**. One can view the nature of the error by calling **cudaGetErrorString()** and printing its output.

For kernel invocations, one can call **cudaGetLastError()**, which does what its name implies. A call would typically have the form

```
cudaError_t err = cudaGetLastError();
if(err != cudaSuccess) printf("%s\n", cudaGetErrorString(err));
```

You may also wish to **cutilSafeCall()**, which is used by wrapping your regular CUDA call. It automatically prints out error messages as above.

Each CUBLAS call (Section [15.18.1](#)) returns a potential error code, of type **cublasStatus**, not checked here.

15.15 Loop Unrolling

Loop unrolling is an old technique used on uniprocessor machines to achieve speedup due to branch elimination and the like. Branches make it difficult to do instruction or data prefetching, so eliminating them may speed things up.

The CUDA compiler provides the programmer with the **unroll** pragma to request loop unrolling. Here an n -iteration **for** loop is changed to k copies of the body of the loop, each working on about n/k iterations. If n and k are known constants, GPU registers may be used to implement the array in the unrolled loop.

For example, the loop

```
for (i = 0; i < 2; i++) {
    sum += x[i];
    sum2 += x[i]*x[i];
}
```

could be unrolled to

```
sum += x[1];
sum2 += x[1]*x[1];
sum += x[2];
sum2 += x[2]*x[2];
```

Here $n = k = 2$. If $\mathbf{x}$ is local to this function, then unrolling will allow the compiler to store it in a register, which could be a great performance enhancer.

The compiler will try to do loop unrolling even if the programmer doesn't request it, but the programmer can try to control things by using the pragma:

```
#pragma unroll k
```

suggest to the compiler a k-fold unrolling. Setting $k = 1$ will instruct the compiler *not* to unroll.

15.16 Short Vectors

In CUDA, there are types such as **int4**, **char2** and so on, up to four elements each. So, an **uint4** type is a set of four unsigned **ints**. These are called **short vectors**.

The key point is that a short vector can be treated as a single word in terms of memory access and GPU instructions. It may be possible to reduce time by a factor of 4 by dividing arrays into chunks of four contiguous words and making short vectors from them.

15.17 Newer Generations

NVIDIA continues to make advances in its GPUs. Many of the advances are of the “bigger and faster than before” type. However, two newer features are more for programmer convenience than speed.

15.17.1 True Caching

Shared memory is in essence a programmer-managed cache, but now on-chip memory can be apportioned to both shared memory and a true cache. This makes less work for the programmer, at a possible cost of reduced performance.

15.17.2 Unified Memory

One of the most significant aspects of GPU programming is writing code to move data back and forth between the CPU memory and the GPU memory. Under the Unified Memory, one can declare some data in one's code to be *managed*, and CUDA will automatically move the data to the proper processor, be it CPU or GPU.

Say for instance a vector **x** is the subject of both CPU and GPU code. If when the CPU code executes **x** is not in CPU memory, the Unified Memory system will copy it from the GPU memory, transparently to the programmer.

Starting with the Pascal model series, there is hardware assist for this, using something similar to virtual memory page tables.

Again, this is for the convenience of the programmer. Hand coding of the memory-to-memory transfers may be much more efficient.

15.18 CUDA from a Higher Level

CUDA programming can involve a lot of work, and one is never sure that one's code is fully efficient. Fortunately, a number of libraries of tight code have been developed for operations that arise often in parallel programming.

You are of course using CUDA code at the bottom, but without explicit kernel calls. And again, remember, the contents of device global memory are persistent across kernel calls in the same application. Therefore you can mix explicit CUDA code and calls to these libraries. Your program might have multiple kernel invocations, some CUDA and others to the libraries, with each using data in device global memory that was written by earlier kernels. In some cases, you may need to do a conversion to get the proper type.

These packages can be deceptively simple. **Remember, each call to a function in these packages involves a CUDA kernel call—with the associated overhead.**

Programming in these libraries is typically much more convenient than in direct CUDA. Note, though, that even though these libraries have been highly optimized for what they are intended to do, they will not generally give you the fastest possible code for any given CUDA application.

We'll discuss a few such libraries in this section.

15.18.1 CUBLAS

CUDA includes some parallel linear algebra routines callable from straight C code. In other words, you can get the benefit of GPU in linear algebra contexts without directly programming in CUDA.

Example: Row Sums Once Again

Below is an example **RowSumsCB.c**, the matrix row sums example again, this time using CUBLAS. We can find the vector of row sums of the matrix A by post-multiplying A by a column vector of all 1s.

I compiled the code by typing

```
gcc -g -I/usr/local/cuda/include -L/usr/local/cuda/lib64 RowSumsCB.c -lcublas
```

You should modify for your own CUDA locations accordingly. Users who merely wish to use CUBLAS will find the above more convenient, but if you are mixing CUDA and CUBLAS, you would use **nvcc**:

```
nvcc -g -G RowSumsCB.c -lcublas
```

Here is the code:

```
1 #include <stdio.h>
2 #include <cublas.h> // required include
3
4 int main(int argc, char **argv)
5 {
6     int n = atoi(argv[1]); // number of matrix rows/cols
7     float *hm, // host matrix
8         *hrs, // host rowsums vector
9         *ones, // 1s vector for multiply
10    *dm, // device matrix
11    *drs; // device rowsums vector
12    // allocate space on host
13    hm = (float *) malloc(n*n*sizeof(float));
14    hrs = (float *) malloc(n*sizeof(float));
15    ones = (float *) malloc(n*sizeof(float));
16    // as a test, fill hm with consecutive integers, but in column-major
17    // order for CUBLAS; also put 1s in ones
18    int i,j;
19    float t = 0.0;
20    for (i = 0; i < n; i++) {
21        ones[i] = 1.0;
22        for (j = 0; j < n; j++)
23            hm[j*n+i] = t++;
24    }
25    cublasInit(); // required init
26    // set up space on the device
27    cublasAlloc(n*n,sizeof(float),(void**)&dm);
28    cublasAlloc(n,sizeof(float),(void**)&drs);
29    // copy data from host to device
30    cublasSetMatrix(n,n,sizeof(float),hm,n,dm,n);
31    cublasSetVector(n,sizeof(float),ones,1,drs,1);
32    // matrix times vector ("mv")
```



```

33     cublasSgemv('n',n,n,1.0,dm,n,drs,1,0.0,drs,1);
34     // copy result back to host
35     cublasGetVector(n,sizeof(float),drs,1,hrs,1);
36     // check results
37     if (n < 20) for (i = 0; i < n; i++) printf("%f\n",hrs[i]);
38     // clean up on device (should call free() on host too)
39     cublasFree(dm);
40     cublasFree(drs);
41     cublasShutdown();
42 }

```

As noted in the comments, CUBLAS assumes FORTRAN-style, i.e. column-major order, for matrices.

Now that you know the basic format of CUDA calls, the CUBLAS versions will look similar. In the call

```
cublasAlloc(n*n,sizeof(float),(void*)&dm);
```

for instance, we are allocating space on the device for an $n \times n$ matrix of **floats**.

The call

```
cublasSetMatrix(n,n,sizeof(float),hm,n,dm,n);
```

is slightly more complicated. Here we are saying that we are copying **hm**, an $n \times n$ matrix of **floats** on the host, to **dm** on the device. The **n** arguments in the last and third-to-last positions again say that the two matrices each have **n** dimensioned rows. This seems redundant, but this is needed in cases of matrix tiling, where the number of rows of a tile would be less than the number of rows of the matrix as a whole.

The 1s in the call

```
cublasSetVector(n,sizeof(float),ones,1,drs,1);
```

are needed for similar reasons. We are saying that in our source vector **ones**, for example, the elements of interest are spaced 1 elements apart, i.e. they are contiguous. But if we wanted our vector to be some row in a matrix with, say, 500 rows, the elements of any particular row of interest would be spaced 500 elements apart, again keeping in mind that column-major order is assumed.

The actual matrix multiplication is done here:

```
cublasSgemv('n',n,n,1.0,dm,n,drs,1,0.0,drs,1);
```

The “mv” in “cublasSgemv” stands for “matrix times vector.” Here the call says: no (‘n’), we do not want the matrix to be transposed; the matrix has **n** rows and **n** columns; we wish the matrix to be multiplied by 1.0 (if 0, this constant multiplication is not actually performed, which we could have specified here); the matrix is at **dm**; the number of dimensioned rows of the matrix is **n**; the vector is at **drs**; the elements of the vector are spaced 1 word apart; we wish the vector to not be multiplied by a scalar (see note above); the resulting vector will be stored at **drs**, 1 word apart.

Further information is available in the CUBLAS manual.

15.18.2 Thrust

The Thrust library is usable not only with CUDA but also to general OpenMP code! So I’ve put my coverage of Thrust in a separate chapter, Chapter ??.

15.18.3 CUFFT

CUFFT does for the Fast Fourier Transform what CUBLAS does for linear algebra, i.e. it provides CUDA-optimized FFT routines.

15.19 Other CUDA Examples in This Book

There are additional CUDA examples in later sections of this book. These include⁸

- Prof. Richard Edgar’s matrix-multiply code, optimized for use of shared memory, Section [20.3.2](#).
- Odd/even transposition sort, Section [10.3.3](#), showing a typical CUDA pattern for iterative algorithms.
- Gaussian elimination for linear systems, Section [20.5.1](#).

⁸If you are reading this presentation on CUDA separately from the book, the book is at <http://heather.cs.ucdavis.edu/~matloff/158/PLN/ParProcBook.pdf>